

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

CO1-AT1-Analytical Problem Solving

Register Number	192311097
Name	DILLILOKESH D

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 40%

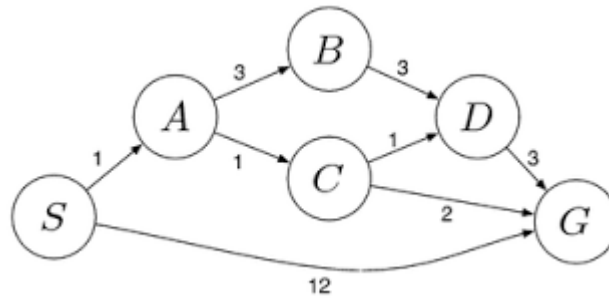
QUESTIONS: 5

Total Marks: 50

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost

path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code of Conduct:

I _____ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

Question 1: Water Jug Problem

Problem: You are given a 4-gallon jug and a 3-gallon jug, neither has measuring markers. There is a pump to fill jugs. Allowed operations: fill jug from pump, pour water out of jug onto ground, pour water from one jug to another. Goal: Obtain exactly 2 gallons of water in the 4-gallon jug.

1. Solution:
2. 1. Fill the 4-gallon jug completely $\rightarrow (4, 0)$
3. 2. Pour from 4-gallon jug to 3-gallon jug (until 3-gallon is full) $\rightarrow (1, 3)$
4. 3. Empty the 3-gallon jug onto the ground $\rightarrow (1, 0)$
5. 4. Pour the remaining 1 gallon from 4-gallon jug to 3-gallon jug $\rightarrow (0, 1)$
6. 5. Fill the 4-gallon jug completely again $\rightarrow (4, 1)$
7. 6. Pour from 4-gallon jug to 3-gallon jug (until 3-gallon is full) $\rightarrow (2, 3)$

Result: The 4-gallon jug now contains exactly 2 gallons of water.

Question 2: Mars Rover Agent Analysis

Percepts for the Agent:

- Visual imagery from stereo panoramic cameras
- Spectral data from spectrometers (rock/soil composition analysis)
- Temperature, pressure, and atmospheric readings
- Wind speed and direction measurements
- Accelerometer and gyroscope data (movement/orientation)
- Position and velocity data (from wheels and sensors)
- Obstacle detection data (from hazard avoidance cameras)
- Electrical power levels (battery/solar panel status)
- Communication signal strength (with Earth/orbiters)

Operating Environment Characteristics:

- **Partially Observable:** Not all terrain is visible; some areas may be occluded
- **Stochastic:** Uncertainty in terrain traction, wheel slippage, unexpected obstacles
- **Sequential:** Current actions affect future states and available options
- **Dynamic:** Environment changes (wind shifts, temperature variations, potential dust storms)
- **Continuous:** State variables (position, orientation, etc.) vary continuously
- **Multi-agent:** May interact with other rovers, orbiters, or mission control

Actions the Agent Can Take:

- Movement: Move forward/backward, turn left/right
- Arm Manipulation: Position robotic arm for sample collection or instrument deployment
- Instrument Activation: Operate cameras, spectrometers, drills, rock abrasion tools
- Communication: Send/receive data to/from Earth or orbiting spacecraft

- Power Management: Adjust solar panel orientation, manage battery usage and consumption
- Diagnostic/System Monitoring: Run self-checks, monitor system health, detect faults

Performance Evaluation Criteria:

- Mission Success: Completion of primary scientific objectives (sample collection, data gathering)
- Data Quality: Quantity and scientific value of collected data
- Efficiency: Distance traveled per unit energy, time to complete objectives

Question 3: 8-Queens Problem Formulation

Problem: Place 8 queens on a chessboard such that none of the queens attack any others.

Problem Formulation as a Search Problem:

- States: **Placements of queens on the board where no two queens attack each other (partial or complete configurations).**
- Initial State: **Empty chessboard (no queens placed).**
- Actions: **Place a queen in an empty square such that it does not attack any already placed queen.**
- Transition Model: **Returns the new board state with the additional queen placed.**
- Goal Test: **True if 8 queens are placed on the board and none attack each other.**
- Path Cost: **Number of queens placed (or uniform cost of 1 per move).**

Representation from the figure description:

- Column 1: Queen at row 1
- Column 8: Queen at row 5 (same diagonal as column 1, row 1)

This is NOT a solution because queens in (1,1) and (8,5) share a diagonal ($|8-1| = |5-1| = 7$).

Question 4: OLA Cab Booking Problem

Problem: A customer wants to travel from one location to another using OLA Cab booking mobile application. The customer can select any cab type: mini, micro, sedan, shared, prime, etc. based on comfort to reach destination.

Problem-Solving Agent Type:

Simple Reflex Agent or Goal-Based Agent (depending on implementation complexity).

Justification: The agent needs to select an appropriate cab type based on user preference (goal) and current state (location, destination). For basic matching based on preferences, a simple reflex agent suffices. For route optimization considering traffic, pricing, etc., a goal-based or utility-based agent would be more appropriate.

Pseudocode for Cab Selection Process:

BEGIN

INPUT: pickup_location, destination, user_preferences (budget, comfort, sharing preference)

OUTPUT: recommended_cab_type

AVAILABLE_CABS $\leftarrow$ get_available_cabs(pickup_location)

FILTERED_CABS $\leftarrow$ FILTER(AVAILABLE_CABS, cab matches user_preferences)

IF FILTERED_CABS is empty THEN

RETURN "No cabs matching preferences"

```

ELSE IF user prefers sharing AND shared cabs available THEN
    RETURN "shared" (lowest cost shared option)
ELSE IF user has budget constraint THEN
    RETURN cheapest cab from FILTERED_CABS that meets budget
ELSE
    RETURN most comfortable cab from FILTERED_CABS (priority: prime > sedan >
micro > mini)
END IF
END

```

Question 5: Logistics Network - Uniform Cost Search

Problem Statement:

A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from starting warehouse S to destination warehouse G using Uniform Cost Search (UCS) algorithm.

Uniform Cost Search Algorithm:

Uniform Cost Search is an optimal search algorithm that expands the node with the lowest path cost from the root. It is both complete and optimal when all step costs are non-negative.

Key characteristics:

- Explores nodes in order of increasing path cost from start
- Uses a priority queue (min-heap) ordered by path cost
- Guarantees to find the least-cost path to goal if one exists
- Equivalent to Dijkstra's algorithm for single-source shortest paths

Pseudocode for UCS:

```

function UNIFORM-COST-SEARCH(problem) returns solution or failure
    return UNIFORM-COST-SEARCH(problem.initial_state, problem.goal_test,
problem.successors)

```

```

function UNIFORM-COST-SEARCH(start, goal_test, successors) returns solution or failure
    frontier ← PRIORITY-QUEUE(start, 0) // Ordered by path cost, lowest first
    explored ← EMPTY-SET()
    while NOT IS-EMPTY(frontier) do
        node ← POP(frontier) // Node with lowest path cost
        if goal_test(node.state) then
            return SOLUTION(node)
        if node.state not in explored then
            add node.state to explored
            for each (action, state, cost) in SUCCESSORS(node.state) do
                child ← NODE(state, node, action, node.path_cost + cost)
                if state not in frontier then
                    INSERT(frontier, child)

```

```

    else if child.path_cost < frontier[state].path_cost then
        REPLACE(frontier, child, child.path_cost)
return failure

```

Application Example:

Consider the following delivery network:

- $S \rightarrow A$ (cost 5), $S \rightarrow B$ (cost 10)
- $A \rightarrow G$ (cost 20), $A \rightarrow C$ (cost 5)
- $B \rightarrow G$ (cost 10), $B \rightarrow C$ (cost 5)
- $C \rightarrow G$ (cost 5)

Execution Steps:

1. Start: S (cost 0)
2. Expand S: Add A(5), B(10) → Frontier: A(5), B(10)
3. Expand A (lowest cost): Add G(25), C(10) → Frontier: B(10), C(10), G(25)
4. Expand C: Add G(15) → Frontier: B(10), G(15), G(25)
5. Expand B: Add G(20) → Frontier: G(15), G(20), G(25), G(25)
6. Expand G (cost 15): GOAL FOUND!
- Path: $S \rightarrow A \rightarrow C \rightarrow G$ with total cost $5 + 5 + 5 = 15$

Optimal Solution Verification:

- $S \rightarrow B \rightarrow G = 10 + 10 = 20$
- $S \rightarrow A \rightarrow G = 5 + 20 = 25$
- $S \rightarrow A \rightarrow C \rightarrow G = 5 + 5 + 5 = 15 \leftarrow \text{OPTIMAL}$

Advantages for Logistics Problem:

- Guarantees optimal (minimum cost) solution
- Naturally handles varying route costs (different fuel/time combinations)
- Complete: Finds solution if one exists
- Efficient for sparse graphs (typical warehouse networks)
- Can incorporate real-time costs (traffic, weather, fuel prices)

Considerations:

- If heuristic data available (e.g., straight-line distance), A* could be more efficient
- For very large networks, consider bidirectional search or hierarchical approaches
- Dynamic costs (changing fuel prices, traffic) would require periodic re-runs or adaptive algorithms

